3-10-2026

Digital System Design and Verification (Tx)

T7

Victor Manuel Sandoval Maciel
Diseño, verificación y validación de sistemas digitales

Content

Instrucción. 2
Desarrollo. 1
Prueba en Tb. 1
Implementación a FPGA 1
Set up de prueba 2
Prueba en FPGA y Teraterm. 2
Prueba con analizador lógico. 3
Tabla de caracteres ASCII 4

Instrucción.

This is a collaborative task. Use Task 6 modules to do an implementation in the FPGA of a UART echo test scenario.

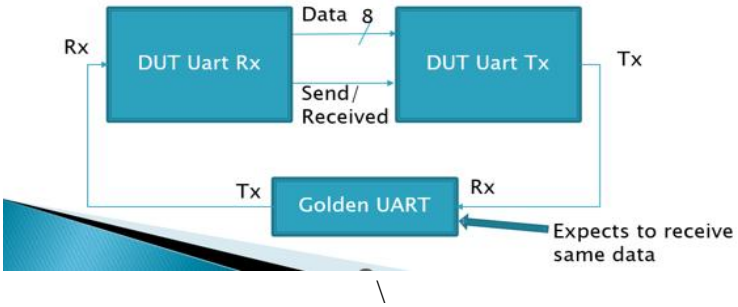
Functional description:

Transmission:

- Clock must run at 5MHz.
- Parallel data input (8-bit data input)
- Send bit (a single clock cycle must be enough to start transmission)
- Architecture must include a PISO register.
- Baud-rate fixed to 115,200 (FPGA implementation only. In simulation counter should be reduced)
- Stop Bit fixed to 2 bits.
- Tx output of 1 bit.
- Ready output when UART is not transmitting.

Implement an echo top module for FPGA:

- Parallel output of receiver connected to parallel input of transmission.
- Received signal of receptor module connected to send input of transmission.
- Send data from PC COM port and receive the same (you can use either Teraterm or python to test that).



Hint: It is important to consider the endianness in the design.
Hint: The use of a clock much faster than baudrate is to fix synchronization between PC and FPGA. Consider restarting Baudrate_generator_counter each time a byte is fully send/received so small sync errors are not accumulated.

Extra points: 20% to create your own test module (sv + py) to test:

- Endianness.
- Received duration
- All possible data sent.
- Correct/incorrect baudrate
- Correct/incorrect parity bit

Deliverables:

- Source files.
- Pictures of test implementation (PC serial port or python script).

Desarrollo.

Se tiene un nuevo top, uart_echo.sv, el cual instancia los módulos desarrollados para Rx_top.sv y uart_tx.sv. En este top se tiene como entrada y salidas:

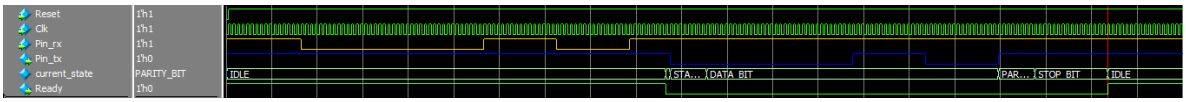
Entradas.

- Reset
- Clk
- Pin_rx

Salidas.

- Pin_tx
- Read

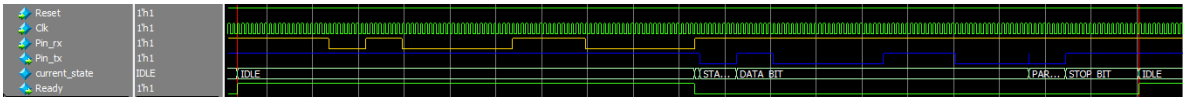
Prueba en Tb.



Se envia un 48 (Cero en ASCII) en binario 0011 0000

Rx recibe (almacena con Parity odd) = 0000 1100 1

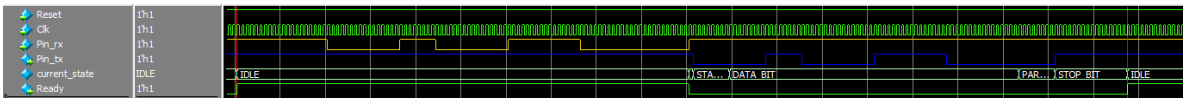
Tx envia (almacena con Parity odd) = 0000 1100 1



Se envia un 49 (Uno en ASCII) en binario 0011 0001

Rx recibe (almacena con Parity odd) = 1000 1100 0

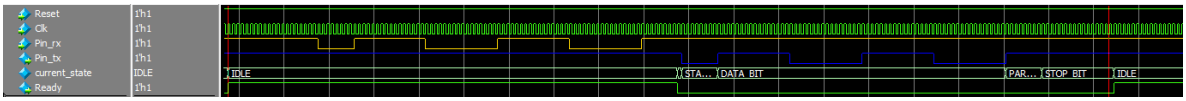
Tx envia (almacena con Parity odd) = 1000 1100 0



Se envia un 50 (dos en ASCII) en binario 0011 0010

Rx recibe (almacena con Parity odd) = 0100 1100 0

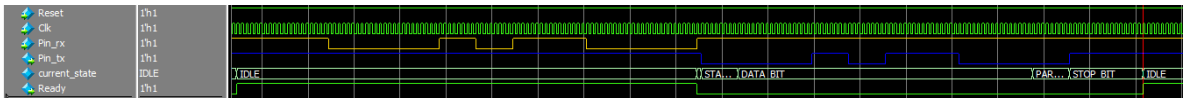
Tx envia (almacena con Parity odd) = 0100 1100 0



Se envia un 51 (tres en ASCII) en binario 0011 0011

Rx recibe (almacena con Parity odd) = 1100 1100 1

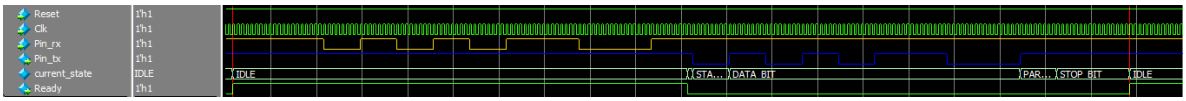
Tx envia (almacena con Parity odd) = 1100 1100 1



Se envia un 52 (Cuatro en ASCII) en binario 0011 0100

Rx recibe (almacena con Parity odd) = 0010 1100 0

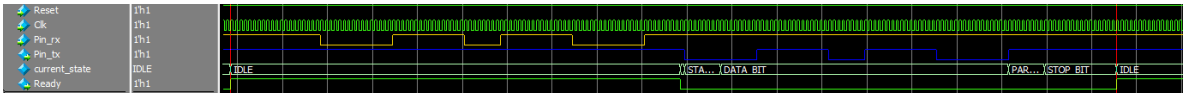
Tx envia (almacena con Parity odd) = 0010 1100 0



Se envia un 53 (Cinco en ASCII) en binario 0011 0101

Rx recibe (almacena con Parity odd) = 1010 1100 1

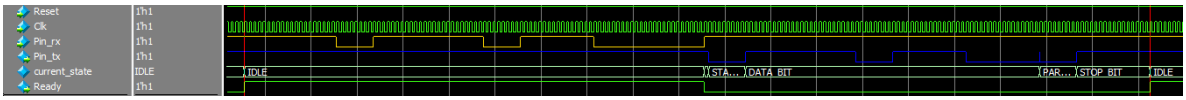
Tx envia (almacena con Parity odd) = 1010 1100 1



Se envia un 54 (Seis en ASCII) en binario 0011 0110

Rx recibe (almacena con Parity odd) = 0110 1100 1

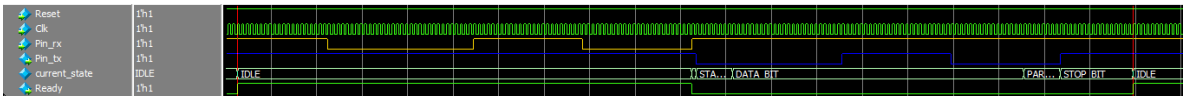
Tx envia (almacena con Parity odd) = 0110 1100 1



Se envia un 55 (Siete en ASCII) en binario 0011 0111

Rx recibe (almacena con Parity odd) = 1110 1100 1

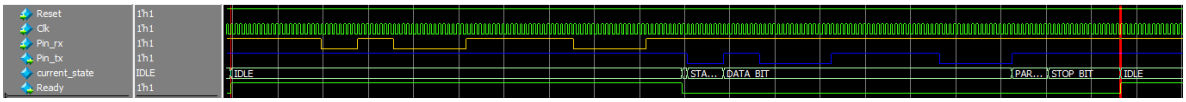
Tx envia (almacena con Parity odd) = 1110 1100 1



Se envia un 56 (Ocho en ASCII) en binario 0011 1000

Rx recibe (almacena con Parity odd) = 0001 1100 1

Tx envia (almacena con Parity odd) = 0001 1100 1



Se envia un 57 (Nueve en ASCII) en binario 0011 1001

Rx recibe (almacena con Parity odd) = 1001 1100 1

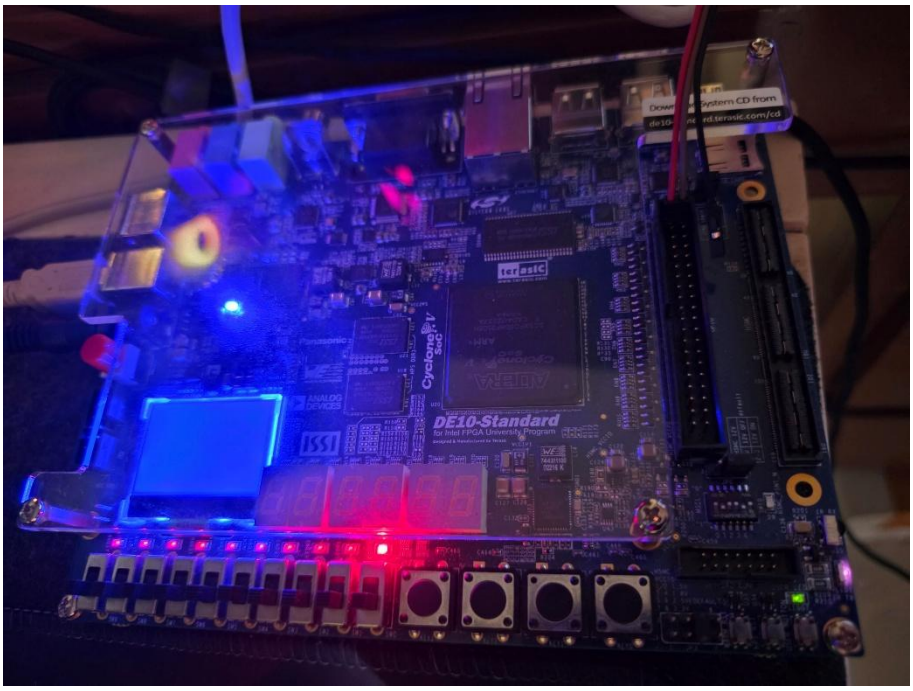
Tx envia (almacena con Parity odd) = 1001 1100 1

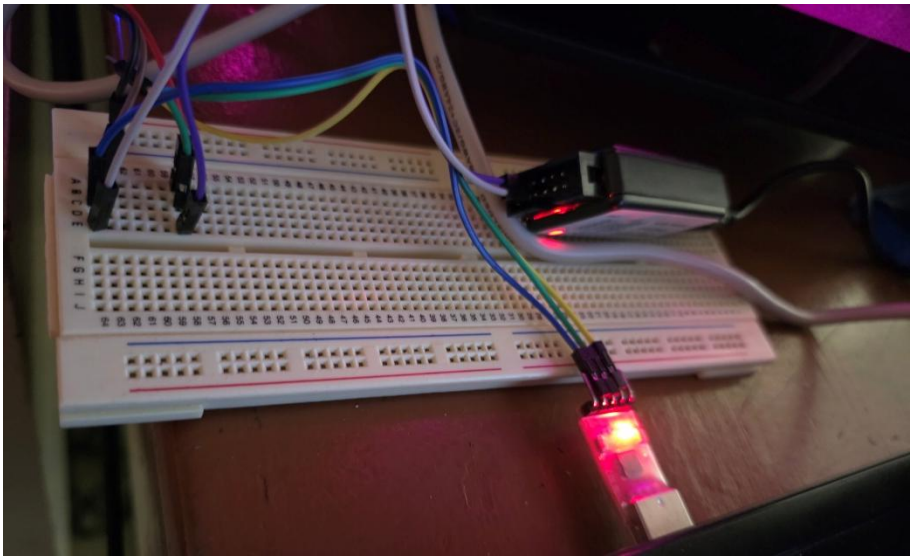
Implementación a FPGA

En donde los pines utilizados para implementación son:

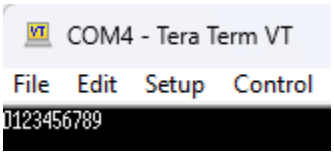
Parte	Pin	Signal
GPIO		
GPIO[0]	PIN_W15	Pin_tx
GPIO[1]	PIN_AK2	Pin_rx
Buttons		
KEY[0]	PIN_AJ4	Reset
LED		
LED0	PIN_AA24	Ready
CLK		
CLOCK_50	PIN_AF14	Clock

Set up de prueba

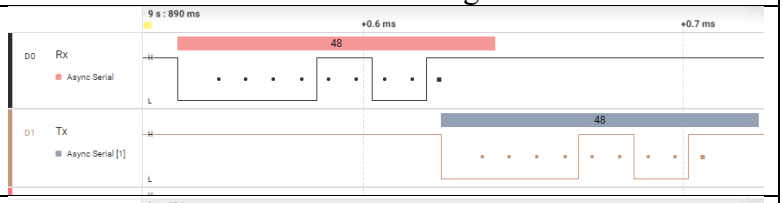
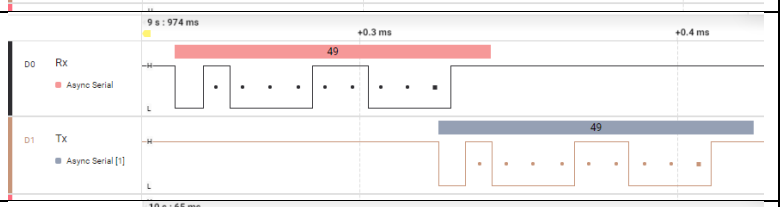
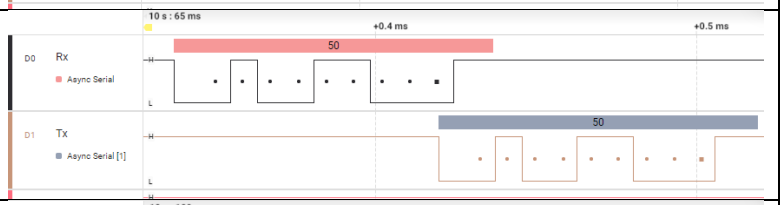
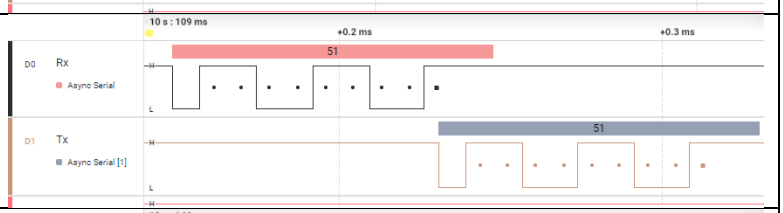
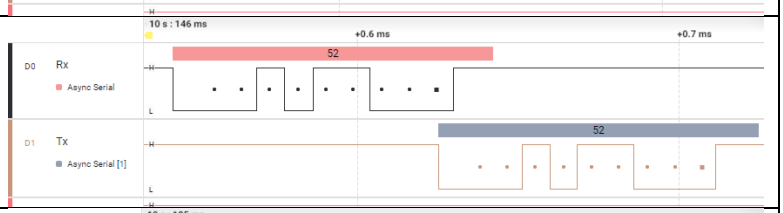
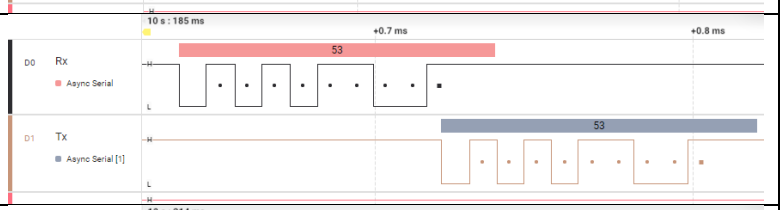
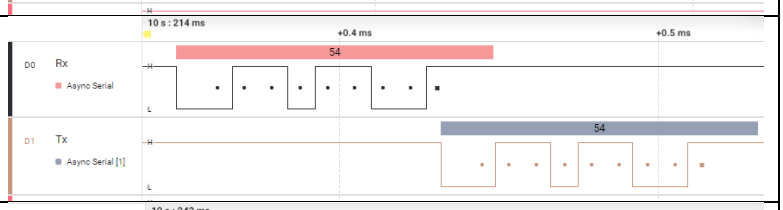
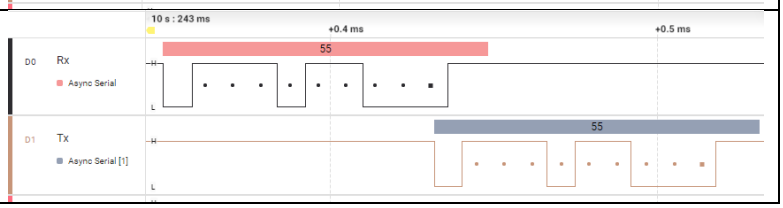




Prueba en FPGA y Teraterm.



Prueba con analizador lógico.

Prueba (Decimal / Assic Code)	En analizador logico
0 = 48	
1 = 49	
2 = 50	
3 = 51	
4 = 52	
5 = 53	
6 = 54	
7 = 55	

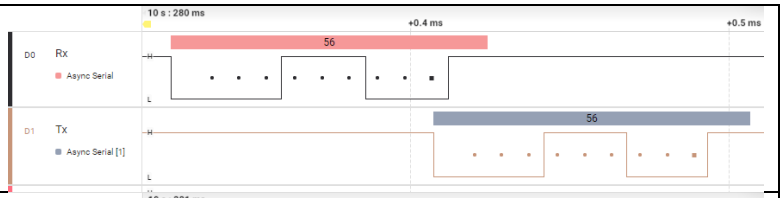
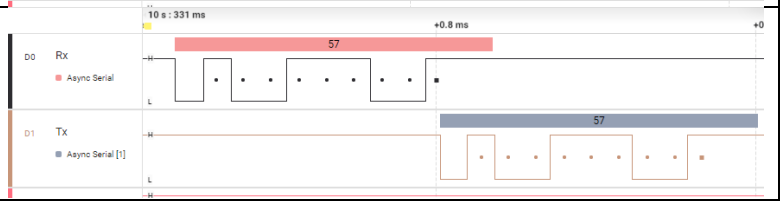
8 = 56	
9 = 57	

Tabla de caracteres ASCII

00	NULL	(carácter nulo)	32	espacio	64	@	96	`	128	Ç	160	á	192	└	224	Ó
01	SOH	(inicio encabezado)	33	!	65	A	97	a	129	ü	161	í	193	┴	225	ß
02	STX	(inicio texto)	34	"	66	B	98	b	130	é	162	ó	194	┬	226	Ô
03	ETX	(fin de texto)	35	#	67	C	99	c	131	â	163	ú	195	├	227	Ò
04	EOT	(fin transmisión)	36	$	68	D	100	d	132	ä	164	ñ	196	─	228	õ
05	ENQ	(consulta)	37	%	69	E	101	e	133	à	165	Ñ	197	┼	229	Õ
06	ACK	(reconocimiento)	38	&	70	F	102	f	134	å	166	ª	198	ã	230	µ
07	BEL	(timbre)	39	'	71	G	103	g	135	ç	167	º	199	Ã	231	þ
08	BS	(retroceso)	40	(	72	H	104	h	136	ê	168	¿	200	╚	232	Þ
09	HT	(tab horizontal)	41	)	73	I	105	i	137	ë	169	®	201	╔	233	Ú
10	LF	(nueva línea)	42	*	74	J	106	j	138	è	170	¬	202	╩	234	Û
11	VT	(tab vertical)	43	+	75	K	107	k	139	ï	171	½	203	╦	235	Ù
12	FF	(nueva página)	44	,	76	L	108	l	140	î	172	¼	204	╠	236	ý
13	CR	(retorno de carro)	45	-	77	M	109	m	141	ì	173	¡	205	═	237	Ý
14	SO	(desplaza afuera)	46	.	78	N	110	n	142	Ä	174	«	206	╬	238	¯
15	SI	(desplaza adentro)	47	/	79	O	111	o	143	Å	175	»	207	¤	239	´
16	DLE	(esc.vínculo datos)	48	0	80	P	112	p	144	É	176	░	208	ð	240	≡
17	DC1	(control disp. 1)	49	1	81	Q	113	q	145	æ	177	▒	209	Ð	241	±
18	DC2	(control disp. 2)	50	2	82	R	114	r	146	Æ	178	▓	210	Ê	242	‗
19	DC3	(control disp. 3)	51	3	83	S	115	s	147	ô	179	│	211	Ë	243	¾
20	DC4	(control disp. 4)	52	4	84	T	116	t	148	ö	180	┤	212	È	244	¶
21	NAK	(conf. negativa)	53	5	85	U	117	u	149	ò	181	Á	213	ı	245	§
22	SYN	(inactividad sínc)	54	6	86	V	118	v	150	û	182	Â	214	Í	246	÷
23	ETB	(fin bloque trans)	55	7	87	W	119	w	151	ù	183	À	215	Î	247	¸
24	CAN	(cancelar)	56	8	88	X	120	x	152	ÿ	184	©	216	Ï	248	°
25	EM	(fin del medio)	57	9	89	Y	121	y	153	Ö	185	╣	217	┘	249	¨
26	SUB	(sustitución)	58	:	90	Z	122	z	154	Ü	186	║	218	┌	250	·
27	ESC	(escape)	59	;	91	[	123	{	155	ø	187	╗	219	█	251	¹
28	FS	(sep. archivos)	60	<	92	\	124	|	156	£	188	╝	220	▄	252	³
29	GS	(sep. grupos)	61	=	93	]	125	}	157	Ø	189	¢	221	¦	253	²
30	RS	(sep. registros)	62	>	94	^	126	~	158	×	190	¥	222	Ì	254	■
31	US	(sep. unidades)	63	?	95	_			159	ƒ	191	┐	223	▀	255	nbsp
127	DEL	(suprimir)														

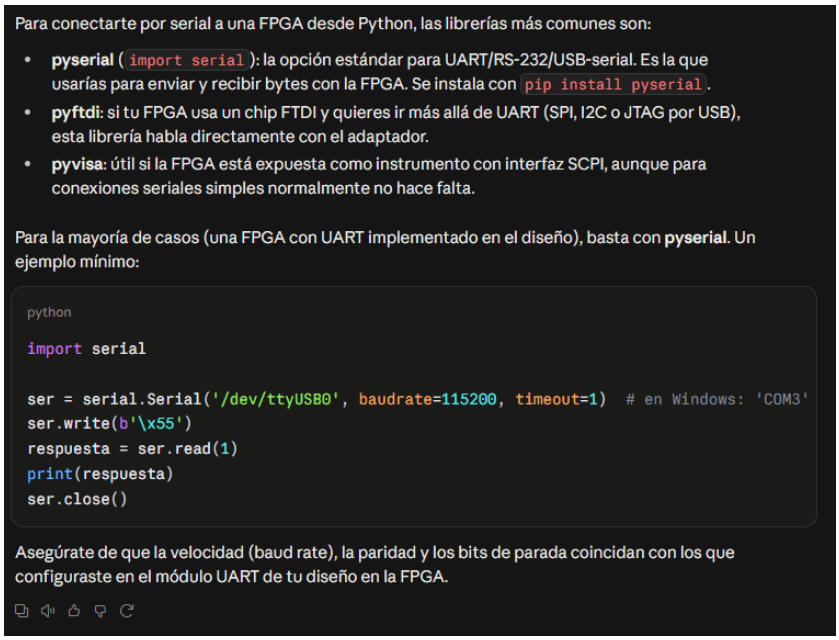
Python tester

Con ayuda de Claude(Haiku 5.5), se tiene un tester echo para la uart.

Prompt

"Usando python, que librerias puedo usar para conectarme por serial a una implementacion de fpga"

Respuesta



Para conectarte por serial a una FPGA desde Python, las librerías más comunes son:

- **pyserial** (`import serial`): la opción estándar para UART/RS-232/USB-serial. Es la que usarías para enviar y recibir bytes con la FPGA. Se instala con `pip install pyserial`.
- **pyftdi**: si tu FPGA usa un chip FTDI y quieres ir más allá de UART (SPI, I2C o JTAG por USB), esta librería habla directamente con el adaptador.
- **pyvisa**: útil si la FPGA está expuesta como instrumento con interfaz SCPI, aunque para conexiones seriales simples normalmente no hace falta.

Para la mayoría de casos (una FPGA con UART implementado en el diseño), basta con **pyserial**. Un ejemplo mínimo:

```python
import serial

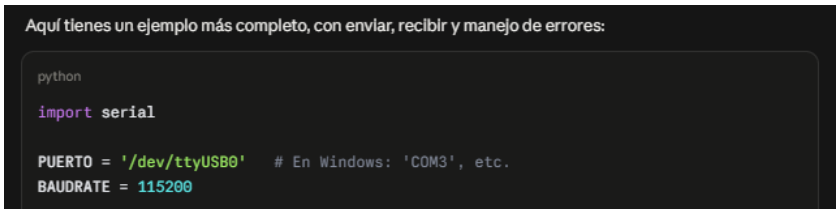
ser = serial.Serial('/dev/ttyUSB0', baudrate=115200, timeout=1)  # en Windows: 'COM3'
ser.write(b'\x55')
respuesta = ser.read(1)
print(respuesta)
ser.close()
```

Asegúrate de que la velocidad (baud rate), la paridad y los bits de parada coincidan con los que configuraste en el módulo UART de tu diseño en la FPGA.

Prompt

Como puedo utilizar import serial. dame un ejemplo

Respuesta.



Aquí tienes un ejemplo más completo, con enviar, recibir y manejo de errores:

```python
import serial

PUERTO = '/dev/ttyUSB0'   # En Windows: 'COM3', etc.
BAUDRATE = 115200
```

```python
try:
    ser = serial.Serial(
        port=PUERTO,
        baudrate=BAUDRATE,
        bytesize=serial.EIGHTBITS,
        parity=serial.PARITY_NONE,
        stopbits=serial.STOPBITS_ONE,
        timeout=1  # segundos para read()
    )

    ser.reset_input_buffer()          # limpia datos viejos
    ser.write(b'\x55')                # envía 1 byte
    respuesta = ser.read(1)           # lee 1 byte (o vacío si hay timeout)

    if respuesta:
        print("Recibido:", respuesta.hex())
    else:
        print("Sin respuesta (timeout)")

except serial.SerialException as e:
    print("Error al abrir o usar el puerto:", e)

finally:
    if 'ser' in locals() and ser.is_open:
        ser.close()
```

Puntos clave:

- `serial.Serial(...)` abre el puerto. Los parámetros deben coincidir con tu módulo UART en la FPGA.
- `write()` recibe bytes (`b'...'`), no strings. Para texto usa `ser.write("hola\n".encode())`.
- `read(n)` espera hasta `n` bytes o hasta que venza el `timeout`.
- `readline()` es útil si tu protocolo termina cada mensaje con `\n`.
- `reset_input_buffer()` evita leer basura que quedó de conexiones previas.

Si me dices qué protocolo o formato de datos usa tu diseño en la FPGA, puedo adaptar el ejemplo.

Con lo que dando un formato para probar todas las tramas posibles se tiene.

Código

```python
import time
import serial

PORT = "COM4"  # Ajustar al puerto de tu FPGA
BAUD = 115200

def main():
    passed = 0
    failed = 0

    with serial.Serial(PORT, BAUD, bytesize=8, parity=serial.PARITY_ODD,
                       stopbits=serial.STOPBITS_TWO, timeout=0.5) as ser:
        for v in range(0, 256):              # todos los valores 0x00 a 0xFF
            ser.reset_input_buffer()
            ser.write(bytes([v]))
            eco = ser.read(1)

            if eco == bytes([v]):
                passed += 1
                print(f"PASS: enviado '{chr(v)}' (0x{v:02X}) -> eco '{chr(eco[0])}' (0x{eco[0]:02X})")
            else:
                failed += 1
                recibido = f"'{chr(eco[0])}' (0x{eco[0]:02X})" if eco else "sin respuesta"
                print(f"FAIL: enviado '{chr(v)}' (0x{v:02X}) -> eco {recibido}")

            time.sleep(0.01)                 # pequeño espacio entre bytes

    print()
    print("======================================")
    print("     UART ECHO TEST FINISHED          ")
    print("======================================")
    print(f" Total   : {passed + failed}")
    print(f" Passed  : {passed}")
    print(f" Failed  : {failed}")
    print("======================================")
    print(" RESULT: ALL TESTS PASSED" if failed == 0 else " RESULT: TEST FAILED")

if __name__ == "__main__":
    main()
```

Esto es una parte de lo que se tiene en terminal

```
PS C:\Users\vitor\Desktop\Verificacion\T7> & C:\Users\vitor\Python313\python.exe c:/Users/vitor/Desktop/Verificacion/T7/python/test.py
PASS: enviado 'j' (0x6A) -> eco 'j' (0x6A)
PASS: enviado 'k' (0x6B) -> eco 'k' (0x6B)
PASS: enviado 'l' (0x6C) -> eco 'l' (0x6C)
PASS: enviado 'm' (0x6D) -> eco 'm' (0x6D)
PASS: enviado 'n' (0x6E) -> eco 'n' (0x6E)
PASS: enviado 'o' (0x6F) -> eco 'o' (0x6F)
PASS: enviado 'p' (0x70) -> eco 'p' (0x70)
PASS: enviado 'q' (0x71) -> eco 'q' (0x71)
PASS: enviado 'r' (0x72) -> eco 'r' (0x72)
PASS: enviado 's' (0x73) -> eco 's' (0x73)
PASS: enviado 't' (0x74) -> eco 't' (0x74)
PASS: enviado 'u' (0x75) -> eco 'u' (0x75)
PASS: enviado 'v' (0x76) -> eco 'v' (0x76)
PASS: enviado 'w' (0x77) -> eco 'w' (0x77)
PASS: enviado 'x' (0x78) -> eco 'x' (0x78)
PASS: enviado 'y' (0x79) -> eco 'y' (0x79)
PASS: enviado 'z' (0x7A) -> eco 'z' (0x7A)
PASS: enviado '{' (0x7B) -> eco '{' (0x7B)
```

Y en el analizador lógico

